Hashing and Collision

Divyashikha Sethia
Divyashikha@dtu.ac.in

Motivation

- Linear search running time: O(n)
- Binary search time O(log n),
  (n: number of elements in array)
- Possible to perform search in O(1) time?

Example

Key	Array of Employees' Records
Key 0 → [0]	Employee record with Emp_ID 0
Key 1 → [1]	Employee record with Emp_ID 1
Key 2 → [2]	Employee record with Emp_ID 2
.....	.....
.....	.....
Key 98 → [98]	Employee record with Emp_ID 98
Key 99 → [99]	Employee record with Emp_ID 99

Example Hashing

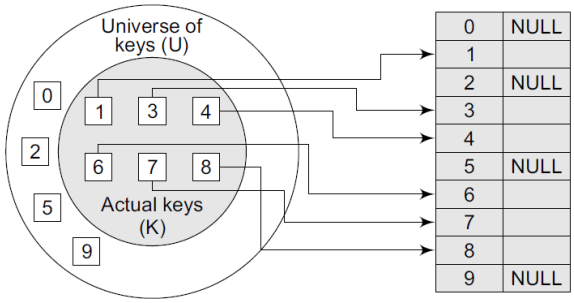
- Only 100 records to keep
- Array size down to tsize that we will actually be using (100 elements), another good option is to use just the last two digits of the key to identify each employee.

Key	Array of Employees' Records
Key 00000 → [0]	Employee record with Emp_ID 00000
.....	.....
Key n → [n]	Employee record with Emp_ID n
.....	.....
Key 99998 → [99998]	Employee record with Emp_ID 99998
Key 99999 → [99999]	Employee record with Emp_ID 99999

Hash Table

- Data structure in which keys are mapped to array positions by hash function.
- Example use hash function that extracts last two digits of key.
- Map keys to array locations or array indices.
- Value stored in a hash table can be searched in O(1) time by using a hash function which generates an address from the key

Hash Table

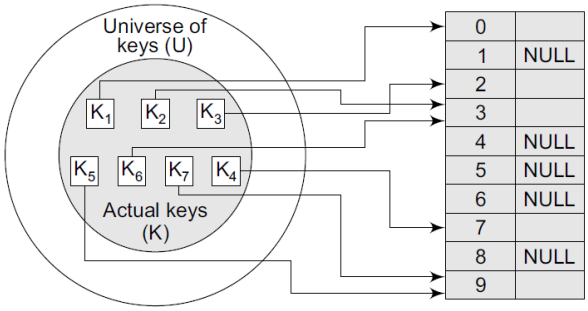


Direct relationship between key and index in the array

Hashing

- An element with key k is stored at index h(k) and not k.
- Hash function h calculates index at which element with key k will be stored.
- Hashing: Process of mapping keys to appropriate locations (or indices) in a hash table.

Hashing



Relationship between keys and hash table index

- keys k2 and k6 point to the same memory location- Collision (two or more keys map to same memory location)

Hash function

- Main goal: reduce range of array indices that have to be handled.
- Instead of having U values, we just need K values, thereby reducing the amount of storage space required.

Hash Function

- Mathematical formula applied to a key, produces an integer which can be used as an index for key in hash table.
- Main aim: Elements should be relatively, randomly, and uniformly distributed.
- Practically no hash function that eliminates collisions completely.
- A good hash function can only minimize the number of collisions by spreading the elements uniformly throughout the array.

Properties of a Good Hash Function

- ***Low cost*** The cost of executing a hash function must be small
- ***Determinism*** A hash procedure must be deterministic. This means that the same hash value must be generated for a given input value.
- ***Uniformity*** A good hash function must map the keys as evenly as possible over its output range.

Hash Functions : Division

- divides x by M and then uses the remainder obtained. In this case, the hash function can be given as h(x) = x mod M
- care should be taken to select a suitable value for M.
- M is an even number then h(x) is even if x is even and h(x) is odd if x is odd. If all possible keys are equi-probable, then this is not a problem. But if even keys are more likely than odd keys, then the division method will not spread the hashed values uniformly

Hash Functions : Division….

- Best to choose M to be a prime number because making M a prime number increases likelihood that keys are mapped with a uniformity in the output range of values.

Hash Functions : Division…

- Drawback: consecutive keys map to consecutive hash values. Although ensures that consecutive keys do not collide, but on the other, it also means that consecutive array locations will be occupied

Hash Functions : Division…

- Setting M = 97, hash values can be calculated as:
- h(1234) = 1234 % 97 = 70
- h(5642) = 5642 % 97 = 16

Hash Functions :Multiplication Method

- *Step 1*: Choose a constant A such that 0 < A < 1.
- *Step 2*: Multiply the key k by A.
- *Step 3*: Extract the fractional part of kA.
- *Step 4*: Multiply the result of Step 3 by the size of hash table (m).

$$h(k) = \lfloor m\ (kA \bmod 1) \rfloor$$

- works practically with any value of A

Hash Functions :Multiplication Method….

- Given a hash table of size 1000, map the key 12345 to an appropriate location in the hash table.

Solution We will use $A = 0.618033$, $m = 1000$, and $k = 12345$

$$
\begin{aligned}
h(12345) &= \lfloor 1000\ (12345 \times 0.618033 \bmod 1) \rfloor \\
&= \lfloor 1000\ (7629.617385 \bmod 1) \rfloor \\
&= \lfloor 1000\ (0.617385) \rfloor \\
&= \lfloor 617.385 \rfloor \\
&= 617
\end{aligned}
$$

Hash Functions :Mid-Square Method

- *Step 1*: Square the value of the key. That is, find k2.
- *Step 2*: Extract the middle r digits of the result obtained in Step 1.

$$h(k) = s$$

where s is obtained by selecting r digits from k^2.

Hash Functions :Mid-Square Method

- Calculate hash value for keys 1234 and 5642 using the mid-square method. The hash table has 100 memory locations

Solution Note that the hash table has 100 memory locations whose indices vary from 0 to 9 This means that only two digits are needed to map the key to a location in the hash table, so `r =`

When `k = 1234`, k^2 `= 1522756, h (1234) = 27`

When `k = 5642`, k^2 `= 31832164, h (5642) = 21`

Observe that the 3rd and 4th digits starting from the right are chosen.

Collision Resolution

- Method to solve the problem of collision:

1. Open addressing
2. Chaining

Collision Resolution by Open Addressing

- Process of examining memory locations in the hash table is called *probing*.
- Open addressing technique can be implemented using linear probing, quadratic probing, double hashing, and rehashing.
- ***Linear Probing***

The simplest approach to resolve a collision is linear probing. In this technique, if a value is already stored at a location generated by `h(k)`, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + i] \bmod m$$

Searching a Value using Linear Probing

- While searching for a value in a hash table, the array index is re-computed and key of element stored at that location is compared with value that has to be searched.
- If match is found, then search operation successful with search time O(1).
- If key does not match, then search function begins a sequential search of the array that continues until: the value is found, or the search function encounters a vacant location in array, indicating that value is not present, or search function terminates because it reaches end of table and value is not present.

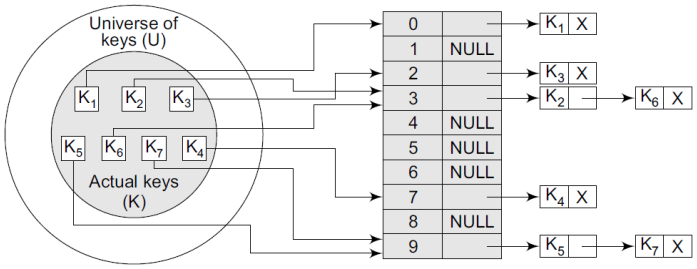
Searching a Value using Linear Probing

- Worst case, search operation may have to make n–1 comparisons, and the running time of the search algorithm may take O(n) time

- Algorithm provides good memory caching through good locality of reference
- Drawback: results in clustering, and hence higher risk of more collisions where one collision has already taken place.
- Performance of linear probing is sensitive to the distribution of input values.

Collision Resolution by Chaining

- Each location in hash table stores pointer to linked list that contains all key values that were hashed to that location.



Searching in chained hash table

- Scanning linked list for entry with given key.
- Insertion operation appends key to end of linked list pointed by hashed location.
- Deleting key: searching list and removing element.
- Widely used due to simplicity to insert, delete, and search a key.
- Cost of inserting a key in a chained hash table: O(1)
- Cost of deleting and searching: O(m) where m is number of elements in list
- Worst case, searching value may take running time of O(n), where n is number of key values stored in chained hash table.

Chained hash table

- **Advantage**: remains effective even when number of key values to be stored is much higher than number of locations in hash table.
- However, with increase in number of keys to be stored, performance does degrade gradually (linearly).
- For example, a chained hash table with 1000 memory locations and 10,000 stored keys will give 5 to 10 times less performance as compared to a chained hash table with 10,000 locations. But a chained hash table is still 1000 times faster than a simple hash table.

Chained hash table

- Unlike quadratic probing, does not degrade when the table is more than half full. This technique is absolutely free from clustering problems and thus provides an efficient mechanism to handle collisions.

Chained hash table

- Inherit disadvantages of linked lists.
- Space overhead with pointer.

Applications

- Used where enormous amounts of data have to be accessed to quickly search and retrieve information.
- **Database indexing**:

- Some database management systems store a separate file known as index file. When data has to be retrieved from file, key information is first searched in appropriate index file which references exact record location of data in database file. This key information in the index file is often stored as a hashed value.

Applications

- **Implement compiler symbol tables in C++**.:

- Compiler uses a symbol table to keep a record of all the user-defined symbols in a C++ program.
- Hashing facilitates the compiler to quickly look up variable names and other attributes associated with symbols.

- **Hashing is also widely used for Internet search engines.**

THANKS